C++ Basics — Chapter 1 Review Notes

Based on *learncpp.com* Chapter 1 (Sections 1.1–1.11)

Chapter theme. Chapter 1 covers the essentials present in *every* C++ program: how source is structured, how objects/variables hold data, how to get input and output, and the vocabulary (statements, expressions, literals, operators, identifiers) you will reuse constantly. Most topics are introduced shallowly here and deepened in later chapters.

Contents

---

1 Statements and the Structure of a Program (1.1)

1.1 Statements

A **statement** is the smallest independent unit of computation in a C++ program — an instruction that tells the program to *do* something. Most (but not all) statements in C++ end with a **semicolon** `;`. A statement written without its terminating semicolon is a common syntax error.

Types of statements you will meet: declaration statements, jump statements, expression statements, compound statements, selection (if/switch), iteration (loops), and try blocks.

1.2 Functions and `main`

A **function** is a collection of statements that execute sequentially. Every C++ program must contain a special function named `main`. When the program is run, execution starts at the top of `main` and proceeds statement by statement, generally top to bottom, until `main` ends (or a return is hit).

```cpp
#include <iostream>  // preprocessor directive: brings in iostream

// main is where execution begins
int main()
{
    std::cout << "Hello, world!";  // statement: print text
    return 0;                      // statement: return status to OS
}
```

1.3 Anatomy of the program

- `#include <iostream>` — a **preprocessor directive** indicating we want to use the contents of the `iostream` library (which contains `std::cout`).
- `int main()` — the function **header**; `int` is the return type, `main` the name, `()` the (empty) parameter list.
- `{ ...}` — the function **body**, a block of statements.
- `return 0;` — returns a **status code** to the operating system. `0` conventionally means success.

Syntax vs. semantics. A **syntax error** breaks the grammar of the language (e.g. a missing ;) and is caught by the compiler. A **semantic error** means the code compiles but does the wrong thing.

2 Comments (1.2)

A **comment** is a programmer-readable note in source code that the compiler ignores. C++ has two forms:

```
// This is a single-line comment. Everything after // is ignored.

/* This is a multi-line comment.
   It spans until the closing delimiter. */

int x { 5 }; // comments can trail a statement
```

- Multi-line comments *cannot be nested* — the first `*/` closes the whole comment.
- **Commenting out code**: temporarily disabling lines by wrapping them in comment syntax — useful for debugging or excluding not-yet-ready code.

Good comment discipline. At the *library/function* level, describe *what* and *why*. Inside a function, describe *why* a particular approach was taken, not the obvious *how*. Comments that merely restate the code add noise.

3 Objects and Variables (1.3)

3.1 Data, values, objects

Data is any information that can be moved, processed, or stored by a computer. A single piece of data is a **value** (also *literal*) — e.g. a letter, a number, or text.

Programs access memory through **objects**: a region of storage that has a value and other properties. A **variable** is a named object — a named piece of memory we use to store a value.

3.2 Definitions and instantiation

To create a variable we use a **definition statement** (also called a **variable definition**):

```
int x; // definition: create an integer variable named x
```

At runtime each defined variable is **instantiated** — it is given an actual memory address where its value lives. An instantiated object is sometimes called an **instance**.

```
int a, b;       // legal: two ints (define multiple of the SAME type)
// int a, y;    // each needs its own type if types differ -> use two lines
double d;       // a floating-point variable
```

3.3 Data types

A **data type** tells the compiler how to interpret the bits of a value into a meaningful value. `int` means the object holds an **integer** (a whole number: 4, 0, -12, etc.). The type is fixed at definition and cannot change.

4 Variable Assignment and Initialization (1.4)

4.1 Copy assignment

Copy assignment uses **operator=** to give an already-created variable a new value:

```
int width;     // definition (no value yet)
width = 5;     // copy assignment: width now holds 5
width = 7;     // reassignment: old value 5 is replaced by 7
```

4.2 Initialization

Initialization gives a variable a value *at the point of definition*. There are several forms:

```
int a;          // (1) default-initialization  -> often indeterminate value
int b = 5;      // (2) copy-initialization
int c ( 6 );    // (3) direct-initialization
int d { 7 };    // (4) direct-list-initialization (brace init) -- PREFERRED
int e {};       // (5) value-initialization / zero-init -> e == 0
int f = { 8 }; // (6) copy-list-initialization
```

Prefer brace (list) initialization `{ }`. It is the modern default because it disallows **narrowing conversions** — attempts to store a value in a type too small to hold it become a compile error instead of a silent data loss.

```
int w { 4.5 }; // ERROR: 4.5 cannot fit into int without narrowing
int v = 4.5;   // compiles, silently truncates to 4 (bad!)
```

4.3 Value-initialization vs. default

- **Value-initialization** with empty braces `{}` sets fundamental types to zero. Use it when you will overwrite the value shortly and don't care about the initial one.
- Prefer initializing variables upon creation. An **unused variable** that is defined but never used will typically trigger a compiler warning.

5 iostream: cout, cin, endl (1.5)

To use `std::cout`, `std::cin`, etc., include the header: `#include <iostream>`.

5.1 `std::cout` and `operator«`

`std::cout` (*character output*) prints to the console using the **insertion operator** `«`. Because each `«` returns the stream, insertions can be **chained**:

```cpp
#include <iostream>

int main()
{
    int age { 21 };
    std::cout << "Age is " << age << " years\n"; // chained insertions
    return 0;
}
```

5.2 `std::endl` vs. `\n`

Both move output to a new line.

- `std::endl` outputs a newline *and* flushes the output buffer (slower).
- `'\n'` outputs a newline without an explicit flush (preferred for performance in most cases).

5.3 `std::cin` and `operator»`

`std::cin` (*character input*) reads keyboard input using the **extraction operator** `»`:

```cpp
#include <iostream>

int main()
{
    std::cout << "Enter a number: ";
    int x {};             // value-initialize before reading
    std::cin >> x;        // read an integer from the user into x
    std::cout << "You entered " << x << '\n';
    return 0;
}
```

Mnemonic for the operators. The angle brackets "point" in the direction data flows: `«` sends data *into* `cout` (toward the console); `»` pulls data *out of* `cin` (toward your variable).

6 Uninitialized Variables and Undefined Behavior (1.6)

6.1 Uninitialized variables

An **uninitialized variable** is one that has not been given a known value. Unlike some languages, C++ does *not* auto-zero most local variables — an uninitialized variable holds whatever **garbage** bits were already at that memory address.

```cpp
int x;              // uninitialized -- value is indeterminate
std::cout << x;     // undefined behavior: prints garbage / may vary
```

6.2 Undefined behavior (UB)

Undefined behavior is the result of executing code whose behavior is not defined by the C++ language. Symptoms include: wrong results, inconsistent results across runs, crashes, or code that

"works" by luck until it doesn't.

Rule. Always initialize your variables. Reading from an uninitialized variable is one of the most common sources of UB for beginners. Related: **implementation-defined behavior** is behavior the standard leaves to the compiler to specify (e.g. the size of `int`).

7 Keywords and Naming Identifiers (1.7)

7.1 Keywords

C++ reserves a set of **keywords** (currently 92) that have special meaning and *cannot* be used as names — e.g. `int`, `return`, `if`, `for`, `class`, `const`, `true`, `false`, `void`, `namespace`. **Special identifiers** such as `override`, `final` are not keywords but have contextual meaning.

7.2 Identifiers

An **identifier** is the name of a variable, function, type, etc. Rules:

- May contain only letters, digits, and underscores.
- Cannot start with a digit (may start with a letter or underscore).
- Cannot be a keyword.
- Are **case-sensitive** (`value` $\neq$ `Value`).

7.3 Naming conventions

- Start variable/function names with a lowercase letter; use `camelCase` or `snake_case` consistently.
- Names beginning with an underscore are conventionally reserved (avoid them).
- Make names descriptive in proportion to scope; the identifier should tell the reader what the value *means*.

8 Whitespace and Basic Formatting (1.8)

Whitespace = spaces, tabs, newlines. C++ is **whitespace-independent** (largely): the compiler ignores most whitespace, so formatting is for *human* readers.

- Two exceptions worth noting: whitespace inside quoted text is preserved literally, and it can be required to separate tokens (`int x` vs `intx`).
- Adjacent string literals separated only by whitespace are concatenated.
- Indent consistently, keep lines readable, and place statements to reflect structure. Consistent style dramatically improves readability and debugging.

```
// These compile identically; the second is readable:
int main(){std::cout<<"Hi";return 0;}

int main()
{
    std::cout << "Hi";
```

```
    return 0;
}
```

9 Literals and Operators (1.9)

9.1 Literals

A **literal** (literal constant) is a fixed value inserted directly into source code — e.g. `5`, `3.14`, `'A'`, `"Hello"`. Unlike a variable, a literal's value cannot change and it has no name.

9.2 Operators

An **operator** is a symbol that performs an operation on one or more inputs called **operands**. Key vocabulary:

- **Arity** = how many operands an operator takes:
  - **Unary** (1 operand): e.g. `-x`.
  - **Binary** (2 operands): e.g. `x + y`, and the stream operators `«` / `»`.
  - **Ternary** (3 operands): the conditional operator `?:`.
- The value produced by an operator is its **return value**.
- Operators can be **chained**, where the output of one feeds the next (`2 * 3 + 4`).

```
int sum { 2 + 3 };       // + is a binary arithmetic operator -> 5
int neg { -sum };        // - here is unary -> -5
bool ok { 4 > 3 };       // > is a binary comparison operator -> true
```

10 Expressions (1.9 cont.)

An **expression** is a combination of literals, variables, operators, and function calls that is evaluated to produce a single value (or performs a side effect). The computed result is the expression's value.

```
2                   // literal expression -> 2
2 + 3               // operator expression -> 5
x                   // variable expression -> value of x
x = 4               // assignment expression -> value assigned
std::cout << x      // function-call-like expression (side effect)
```

10.1 Expression statements

An **expression statement** is an expression followed by a semicolon — it turns an expression into a standalone statement:

```
x = 5;              // expression `x = 5` made into a statement
std::cout << x;     // expression statement (side effect: printing)
```

Distinction to hold onto. A **statement** is a complete instruction (often ends in ;). An **expression** is evaluated for a value and by itself is *not* a complete statement — append ; to make it an expression statement.

11 Developing Your First Program (1.10–1.11)

11.1 Putting it together

A first complete interactive program combines everything above: include `iostream`, define `main`, prompt with `std::cout`, read with `std::cin`, compute with operators, and print the result.

```
#include <iostream>

int main()
{
    std::cout << "Enter an integer: ";
    int num {};                  // value-initialize
    std::cin >> num;             // read input

    std::cout << num << " doubled is: "
              << num * 2 << '\n';  // expression in output
    return 0;
}
```

11.2 Iterative development & good habits

- Write a little, compile, and run often — catch errors while the change set is small.
- Keep programs simple first; add complexity incrementally.
- Don't fear the compiler's errors/warnings — read them; they point at the fix.
- Learn to use the debugger early rather than relying on print statements alone.

12 Key Terms — Quick Glossary

Statement
: Smallest independent instruction; usually ends in ;.

Function
: A named sequence of statements; `main` is the entry point.

Comment
: Programmer note ignored by the compiler (// or /* */).

Data / Value
: Information processed by a program; a single piece is a value.

Object
: A region of storage holding a value.

Variable
: A named object.

Definition
Statement that creates a variable.

Instantiation
Runtime assignment of a memory address to a variable.

Data type
Tells the compiler how to interpret an object's bits.

Copy assignment
Using `operator=` to change an existing variable's value.

Initialization
Providing a value at the point of definition.

Brace initialization
`{ }` form; forbids narrowing conversions (preferred).

Narrowing conversion
A conversion that may lose data; brace-init rejects it.

std::cout / «
Console output stream and the insertion operator.

std::cin / »
Console input stream and the extraction operator.

std::endl
Newline plus buffer flush (`'\n'` avoids the flush).

Uninitialized variable
Variable with no known value; reading it is UB.

Undefined behavior
Executing code the standard does not define.

Keyword
Reserved word with special meaning; not usable as a name.

Identifier
The name of a variable, function, or type (case-sensitive).

Whitespace
Spaces/tabs/newlines; mostly ignored, used for readability.

Literal
A fixed value written directly in code.

Operator / Operand
A symbol performing an operation on its inputs.

Arity
Number of operands: unary, binary, or ternary.

Expression
Combination evaluated to a single value.

Expression statement
An expression plus ;.

End of Chapter 1 review. Next: Chapter 2 — C++ Basics: Functions and Files.